

Antarctica Regional Weather Forecasting

Model Configuration, Station Breakdowns, and Remote Execution Guide

1. Executive Summary

This project implements a regionalized **FourCastNet (Adaptive Fourier Neural Operator - AFNO)** framework for high-accuracy weather forecasting at two Indian research stations in Antarctica: **Bharati** and **Maitri**. To prevent local hardware crashes and ensure absolute training stability, a single-threaded CPU execution pipeline with automated cooling intervals is utilized. Additionally, a machine learning post-processing layer (Ridge Regression Correctors) resolves localized systematic biases dynamically.

2. Model Architecture & Common Parameters

The regional FourCastNet neural network uses the following configuration parameters:

Parameter Name	Value	Description
Lookback Window (IN_STEPS)	8 steps (48 hours)	Historical duration fed into the model.
Prediction Window (OUT_STEPS)	4 steps (24 hours)	Direct forecasting horizon for the neural network.
Time Interval	6 hours	Temporal resolution of the ERA5 dataset.
Patch Size	4 x 4 pixels	Size of spatial patches mapped to the embedding space.
Embedding Dimension	128	Dimensionality of features in the transformer layers.
Number of Blocks	6	Number of sequential AFNO blocks in the model depth.
MLP Ratio	4	Expansion multiplier inside the block feedforward networks.
Learning Rate	1e-4	Initial learning rate utilizing Cosine Annealing scheduler.
Epochs	30	Maximum training epochs with 10-epoch early stopping patience.

3. Station Grid Specific Parameters

The spatial datasets are customized for each station's respective geographical bounding boxes:

Parameter	Bharati Station	Maitri Station
Coordinates	69.01° S, 76.19° E	70.76° S, 11.73° E
Bounding Box (Lat)	-60.0° S to -89.75° S	-68.0° S to -73.0° S
Bounding Box (Lon)	50.0° E to 99.75° E	0.0° E to 20.0° E
Grid Resolution	120 x 200 points	21 x 81 points
Padded Resolution	120 x 200 points	24 x 84 points
Station Index (Lat, Lon)	Index (36, 105)	Index (11, 47)
Dataset File	era5_processed.npy	era5_maitri.npy
Raw GRIB size	10.42 GB	362 MB
Processed Dataset size	5.61 GB (14,612 steps)	495 MB (13,860 steps)
Dataset Variables	u10, v10, t2m, msl	t2m, u10, v10, z
Split logic	Train: 2016-23 BC: 2023 Test: 2024-25	Train: 2017-24 BC: 2024 Test: 2025-26

4. Historical Out-of-Sample Performance

Post-processing via non-linear Ridge regression correctors resolves systematic diurnal and seasonal errors, yielding significant R^2 gains:

Station	Variable	Raw Model R^2	Bias-Corrected R^2	Evaluation Period
Bharati	2m Temperature	0.908	0.923	Years 2024 - 2025
Bharati	10m Wind Speed	0.814	0.835	Years 2024 - 2025
Maitri	2m Temperature	0.899	0.911	Years 2025 - 2026
Maitri	10m Wind Speed	0.767	0.795	Years 2025 - 2026

5. Step-by-Step Remote Execution Guide

To migrate and run the model training and evaluation pipeline on a remote CPU server (e.g., Linux-based server), follow this checklist:

Step 5.1: Package and Compress the Repository

Compress only the source code and configuration files. Do NOT include raw datasets, checkpoints, or virtual environments to minimize transfer size. Run this in your local command prompt:

```
tar --exclude='.venv' --exclude='data/raw/*.grib' --exclude='*.npy' -czf
antarctica_forecast.tar.gz .
```

Step 5.2: Transfer Package to Remote Server

Use Secure Copy Protocol (SCP) to upload the archive to your target server:

```
scp antarctica_forecast.tar.gz username@remote_host:/path/to/destination
```

Step 5.3: Extract and Set Up Environment

Connect to the remote host, extract the files, and set up a python virtual environment:

```
ssh username@remote_host
cd /path/to/destination
tar -xzf antarctica_forecast.tar.gz
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Step 5.4: Execute the Training Pipeline

To run the training sequentially under safe single-core CPU parameters, execute the following scripts in order:

```
# Preprocess raw GRIB data into Numpy arrays
python3 src/preprocess_maitri_safe.py

# Train the base neural network on CPU (saves best checkpoint)
python3 src/train.py

# Train bias-correction estimators on dynamic splits
python3 src/train_bias_correction.py

# Generate out-of-sample visual validation dashboards
python3 src/generate_dashboards.py
```